

INT315 • CLUSTER COMPUTING

UNIT I

Introduction to Apache Spark

From MapReduce's Limitations to a Working Local Spark Cluster

Theory -> Comic Visuals -> Industry Context -> Commands/Code -> Self-Assessment

Six syllabus topics, fully covered end to end, with a closing quiz and hands-on exercise

Session 2026-27

Companion Study Material — INT315 Cluster Computing, Unit I

Table of Contents

Unit I at a Glance	3
1. Limitations of MapReduce in Hadoop	4
1.1 The Disk-Bound Intermediate Storage Bottleneck	4
1.2 High Latency.....	5
1.3 Verbosity of Code.....	5
2. Batch vs. Real-Time Analytics	7
2.1 Definitions & Latency Trade-offs	7
2.2 Lambda vs. Kappa Architecture	7
3. Applications of Stream Processing & In-Memory Processing	10
3.1 How RAM-Based Computing Changes Big Data Processing.....	10
3.2 Real-World Use Cases	10
4. Features & Benefits of Apache Spark	13
4.1 Speed	13
4.2 Ease of Use.....	13
4.3 Unification — SQL, Streaming, ML, and Graphs in One Engine	13
4.4 Lazy Evaluation	14
5. Local / Standalone Installation & Environment Setup.....	17
5.1 Prerequisites	17
5.2 Downloading & Extracting Spark	17
5.3 Setting Environment Variables	17
5.4 Starting & Stopping the Standalone Cluster	18
5.5 Verifying the Web UIs	19
6. Spark vs. Hadoop Ecosystem	21
Unit I Review	23

Unit I at a Glance

This unit answers one central question: why did the Big Data world move from Hadoop MapReduce to Apache Spark, and what exactly does Spark bring to the table? We'll trace the problem (MapReduce's disk-bound bottleneck), the broader shift it's part of (batch vs. real-time analytics), the underlying enabler (in-memory computing), the tool itself (Spark's features and unified stack), how to actually stand up a working cluster on your own machine, and finally a precise, industry-standard comparison against the Hadoop ecosystem as a whole.

Every topic below follows the same five-beat rhythm: Theory & Mechanics, a Comic/Visual metaphor, Industry Context, Commands/Code you can run yourself, and a Self-Assessment. The unit closes with a consolidated 3-question review quiz and one hands-on exercise.

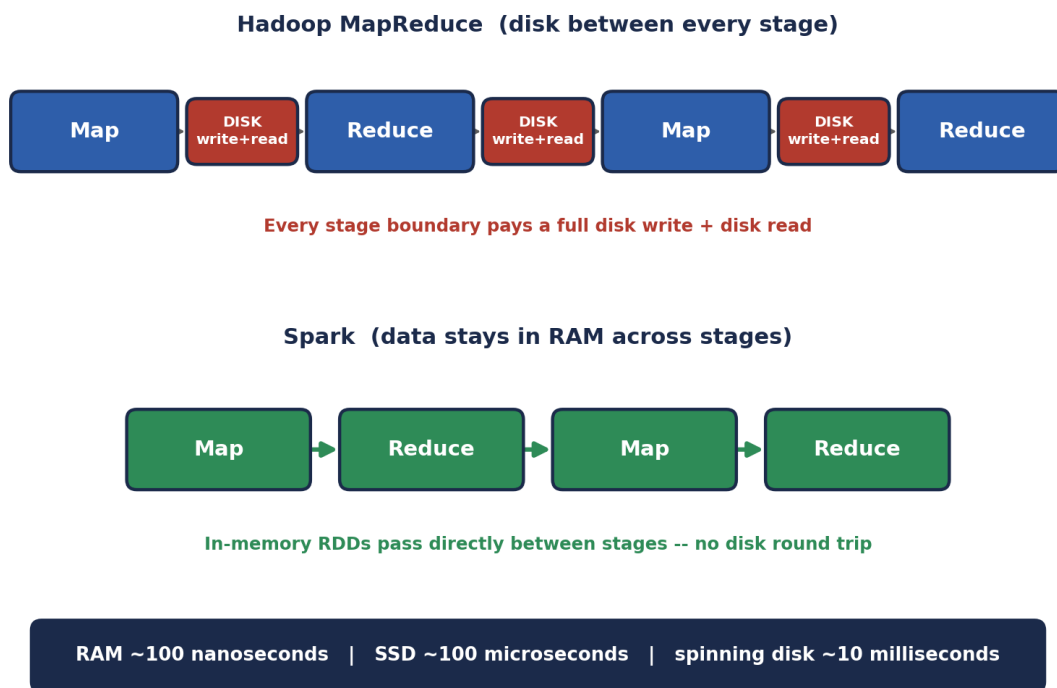
1. Limitations of MapReduce in Hadoop

Apache Spark wasn't invented in a vacuum — it was a direct engineering response to three well-understood pain points in Hadoop MapReduce that real teams hit constantly in production. Understanding these three limitations precisely is the foundation for everything else in this unit.

1.1 The Disk-Bound Intermediate Storage Bottleneck

Classic MapReduce writes its intermediate results to disk after every single Map and Reduce stage — not as an accident, but as a deliberate fault-tolerance choice: if a machine crashes mid-job, the already-completed stage's output survives on disk and doesn't need to be recomputed. The cost of that safety is speed. Every stage boundary pays the price of a full disk write followed by a full disk read, and for any job that chains multiple MapReduce stages together — which describes most real analytical pipelines — that cost compounds stage after stage.

Why MapReduce Is Slow: The Disk-Bound Intermediate Storage Bottleneck



Why MapReduce Is Slow: The Disk-Bound Intermediate Storage Bottleneck

The underlying reason this hurts so much is a simple hardware fact: RAM access takes roughly 100 nanoseconds, an SSD read takes roughly 100 microseconds — about 1,000x slower — and a spinning disk seek takes roughly 10 milliseconds — about 100,000x slower than RAM. MapReduce's design forces every multi-stage job through the slowest tiers of that hierarchy, again and again, whether the job actually needs that level of durability or not.

1.2 High Latency

Disk I/O is only part of the latency story. Every MapReduce job also pays a real startup cost: the JobTracker (or, in YARN, the ApplicationMaster) has to negotiate resources, launch fresh JVM processes for each task, and only then begin actual computation. For a short job, this startup overhead can dwarf the actual useful work being done. Historically, launching even a trivial MapReduce job commonly took tens of seconds before a single byte of real processing happened — utterly unworkable for interactive queries or anything resembling a real-time response.



What this meant in practice

Data analysts running exploratory Hive-on-MapReduce queries in the mid-2010s routinely waited minutes for even simple aggregate queries to return — not because the underlying data was large, but because of MapReduce's per-job startup and per-stage disk overhead. This directly motivated the creation of faster query engines (Spark SQL, Impala, Presto) that could execute the same SQL far faster on the same data.

1.3 Verbosity of Code

Beyond raw performance, MapReduce imposes a real productivity cost: even a simple word-count program requires writing a full Mapper class, a full Reducer class, and a Driver program to wire them together — typically 50-60+ lines of Java boilerplate for a task that is conceptually two or three lines of logic. Compare the same word-count problem in classic Hadoop MapReduce Java versus Spark's Scala or Python API:

```
// ---- Classic Hadoop MapReduce word count (abbreviated Java) ----
public class WordCount {
    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }
}

public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) sum += val.get();
        context.write(key, new IntWritable(sum));
    }
}
```

```

    }
}
public static void main(String[] args) throws Exception {
    // ... 15+ more lines configuring Job, InputFormat, OutputFormat ...
}
}

# ---- The SAME word count in PySpark ----
text = sc.textFile("hdfs:///data/input.txt")
counts = (text.flatMap(lambda line: line.split())
           .map(lambda word: (word, 1))
           .reduceByKey(lambda a, b: a + b))
counts.saveAsTextFile("hdfs:///data/output")

```

Ordering a Coffee vs. Filing a Building Permit

Writing a MapReduce job for a simple task feels like filing paperwork to build a small shed -- you fill out a Mapper form, a Reducer form, a Driver form, specify InputFormat and OutputFormat in triplicate, and only THEN are you allowed to actually build anything.

Writing the same task in Spark feels like ordering a coffee -- you say what you want ("split these lines, count these words") in a few words, and the barista (Spark's engine) handles everything else -- the equivalent of building permits, staffing, and scheduling -- invisibly behind the counter.

Neither approach is 'wrong' -- the building-permit process exists for real safety reasons (explicit fault tolerance, explicit resource control). But for the vast majority of everyday data tasks, the coffee-order experience is what actually gets used in practice today.

Check yourself — Section 1

Q1. Why does classic MapReduce write intermediate data to disk between every stage, and what does it gain by doing so?

Q2. Roughly how much slower is a spinning disk seek than a RAM access, and why does that matter for multi-stage jobs?

Q3. Besides raw execution speed, name one other practical cost of using classic MapReduce that Spark specifically reduces.

Hands-on Challenge:

Find (or write from memory) a Hadoop MapReduce word-count Java program online, and count its total lines of code including imports and boilerplate. Compare that number directly against the 5-line PySpark version above, and write one sentence on what that difference means for a team's development speed.

2. Batch vs. Real-Time Analytics

With MapReduce's limitations established, the next question is architectural: how should a data platform actually be organized to serve both slow, thorough analysis AND fast, immediate answers? This section defines the two processing models precisely, then compares two real architectural patterns for combining them.

2.1 Definitions & Latency Trade-offs

- Batch processing — data is collected over a period of time (an hour, a day) and processed as one finite, bounded dataset, all at once. Latency from an event happening to an insight being available is typically minutes to hours. Optimized for throughput and completeness, not speed.
- Real-time (stream) processing — data is processed continuously, record by record or in small micro-batches, as it arrives. Latency is measured in milliseconds to seconds. Optimized for freshness, often at some cost to completeness (a stream processor may not have seen 100% of "late" data yet when it produces an answer).

Neither approach is universally better — they trade completeness and throughput against freshness. A monthly financial reconciliation report is naturally a batch job: correctness and completeness matter far more than getting an answer in the next five seconds. Fraud detection on a live credit card swipe needs an answer within milliseconds, or the fraudulent transaction has already gone through by the time a batch job would have caught it.

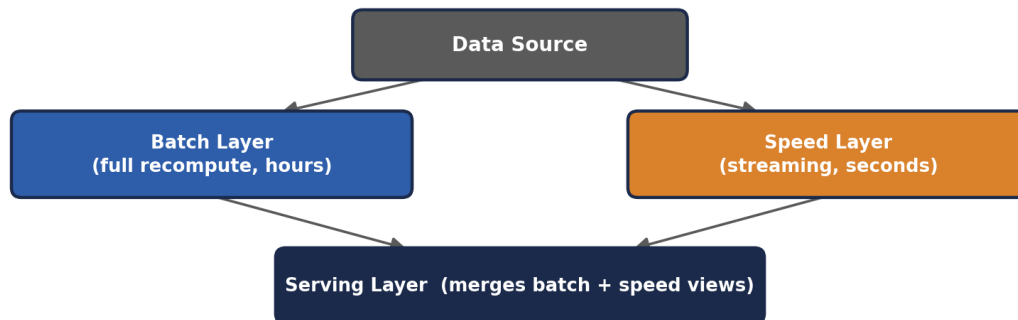
2.2 Lambda vs. Kappa Architecture

Once an organization needs BOTH batch accuracy and real-time freshness from the same underlying data, two competing architectural patterns emerged for how to structure that system.

- Lambda Architecture — runs two separate, parallel pipelines against the same incoming data: a Batch Layer that periodically recomputes complete, accurate results from full history, and a Speed Layer that processes only the most recent data with low latency to fill the gap until the next batch run catches up. A Serving Layer merges both views for querying.
- Kappa Architecture — a simplification that uses ONE streaming pipeline for everything. Instead of maintaining a separate batch codebase, historical reprocessing is handled by simply replaying the stream's log from an earlier point in time through the same streaming engine.

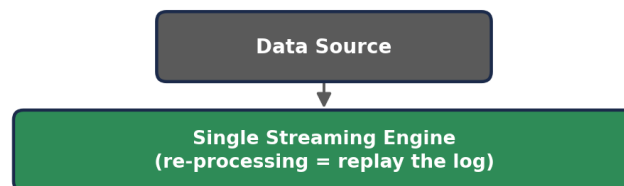
Lambda vs. Kappa Architecture — Two Ways to Combine Batch + Real-Time

Lambda Architecture — two separate paths



Two codebases to build and keep in sync -- real operational cost

Kappa Architecture — one unified streaming path



One codebase handles both real-time AND historical reprocessing

Lambda vs. Kappa Architecture — Two Ways to Combine Batch + Real-Time

Lambda's honest weakness is visible right in its diagram: the batch logic and the streaming logic are two DIFFERENT codebases that have to be built, tested, and kept behaviorally consistent with each other — a real, ongoing engineering cost. Kappa's appeal is collapsing that down to one codebase, made practical specifically because modern stream processors (Kafka combined with Spark Structured Streaming, in particular) can replay historical data through the exact same streaming logic used for live data, removing the need for a separate batch path.

Which pattern is actually used today

Lambda architecture was the dominant pattern through the early-to-mid 2010s, when stream processors weren't yet mature or fast enough to handle full historical reprocessing reliably. As Kafka's log-retention model and Spark Structured Streaming matured, many organizations have shifted toward Kappa-style designs specifically to eliminate the dual-codebase maintenance burden — though Lambda still appears in systems where the batch and speed layers genuinely need different processing logic, not just different latency.

 Check yourself — Section 2

Q1. In your own words, what's the fundamental latency vs. completeness trade-off between batch and real-time processing?

Q2. What specific engineering cost does Lambda Architecture incur that Kappa Architecture avoids?

Q3. Give one business scenario where batch processing is clearly the right choice, and one where real-time is clearly required.

Hands-on Challenge:

Sketch your own two-column comparison (batch vs. streaming) for THREE example use cases of your choosing — not the ones already used in this unit — and justify each choice in one sentence.

3. Applications of Stream Processing & In-Memory Processing

This section connects the dots between Section 1's disk bottleneck and Section 2's real-time requirement: in-memory computing is the specific enabling technology that makes true low-latency stream processing possible at all.

3.1 How RAM-Based Computing Changes Big Data Processing

When intermediate data stays in RAM instead of round-tripping to disk between processing steps, two things change simultaneously: raw throughput goes up (RAM is 1,000-100,000x faster than disk, per Section 1.1), and — just as importantly — LATENCY becomes predictable and small enough to support sub-second responses. Disk-based systems can be fast on average but are prone to occasional large latency spikes (a slow disk seek, a filesystem hiccup); RAM-resident processing avoids that entire class of unpredictability, which matters enormously for real-time systems where a single slow response can mean a missed fraud window or a stale dashboard.

Spark exploits this by keeping RDD partitions cached in Executor memory across an application's lifetime (Prerequisites: Executors are long-lived, not re-launched per task), and Structured Streaming extends this same in-memory execution model to continuously arriving data, processed in small in-memory micro-batches rather than being flushed to disk between each one.

3.2 Real-World Use Cases

- Fraud detection — a card swipe or online transaction must be scored against a machine learning model and either approved or blocked within a strict latency budget (commonly under a few hundred milliseconds), or the transaction completes before a fraud signal can act on it.
- Real-time dashboards — operations teams monitoring live metrics (server health, order volume, sensor readings) need dashboards that reflect what's happening NOW, not what happened in last night's batch job.
- Recommendation engines — streaming platforms update "recommended for you" content within seconds of a user's last click or watch, using their most recent behavior rather than yesterday's batch-computed profile.
- IoT and sensor monitoring — factories, vehicles, and infrastructure emit continuous telemetry streams that must be monitored for anomalies (equipment failure, safety thresholds) in near real-time.

Real-Time Fraud Detection — A Stream Processing Pipeline



end-to-end latency target: under ~200 milliseconds, swipe to decision

This is exactly the shape of the industry cases in Module 3 below:
real-time dashboards and IoT alerting follow the identical pattern.

Real-Time Fraud Detection — A Stream Processing Pipeline



The Difference Between a Security Guard and a Detective

Batch processing is a detective reviewing yesterday's security footage in detail -- thorough, accurate, but the burglar is long gone by the time the report is filed. Stream processing with in-memory computing is a security guard watching a live monitor RIGHT NOW -- less time to deliberate, but able to actually intervene while something is still happening.

A mature data platform needs both roles on staff: the detective (batch) building the complete, trustworthy historical record, and the guard (streaming) catching what needs an immediate response. Neither role replaces the other.



Why this drives real hiring and tooling decisions

Companies building fraud systems, ad-bidding platforms, or live operational dashboards specifically hire for "streaming" or "real-time data engineering" skills distinct from general "batch ETL" roles, precisely because the latency and reliability requirements demand different tools (Kafka, Spark Structured Streaming, Flink) and a different mental model (continuous, unbounded data) than traditional batch pipelines.



Check yourself — Section 3

Q1. Why does keeping data in RAM reduce not just average latency but also latency UNPREDICTABILITY?

Q2. Pick one of the four use cases above and explain, in your own words, what would go wrong if it were implemented as a nightly batch job instead.

Q3. How does Spark's long-lived Executor model (from Section 1) directly support both fast batch AND real-time streaming workloads?

Hands-on Challenge:

Design (on paper) a simple real-time use case of your own -- not one already listed -- and sketch its pipeline as a sequence of 4-5 boxes, the way the fraud detection diagram above does, labeling each stage's role.

4. Features & Benefits of Apache Spark

With the motivation fully established, this section introduces Spark itself: what it actually is, and the specific features that make it the dominant modern processing engine.

4.1 Speed

Spark's headline feature, covered in depth in Section 1: in-memory RDD execution across stages, avoiding the disk round-trips that define classic MapReduce. Independent benchmarks and real production comparisons commonly show 10-100x speedups on iterative and multi-stage workloads, with the largest gains on workloads that reuse the same data repeatedly — exactly the shape of machine learning training and interactive analytics.

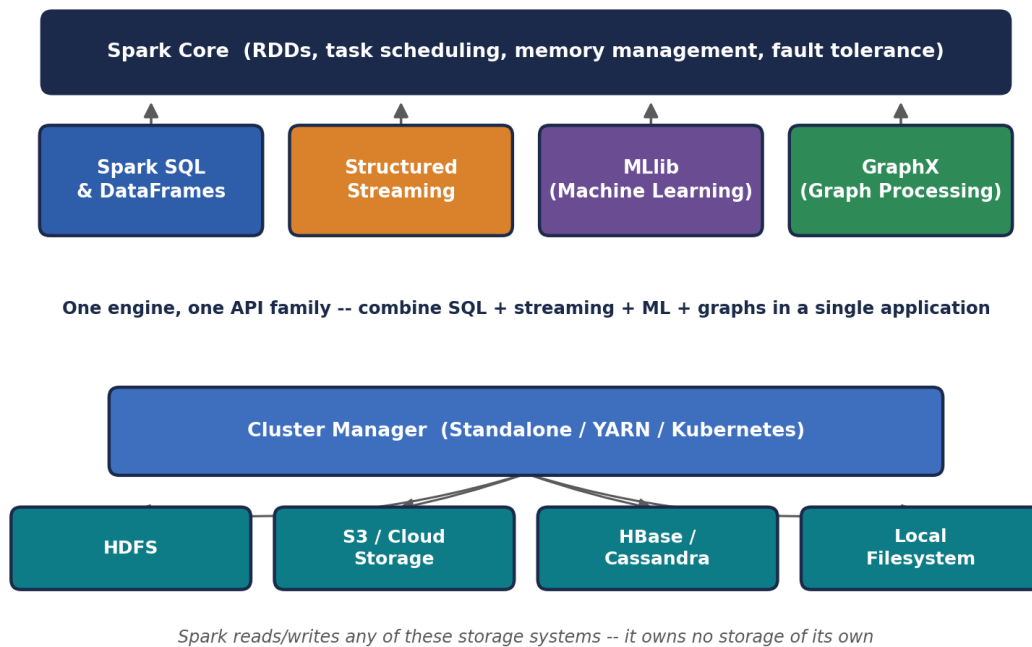
4.2 Ease of Use

Spark exposes the same core capabilities through APIs in Scala, Python (PySpark), Java, R, and SQL, letting teams work in whichever language fits their skills and their problem — a data scientist comfortable in Python and a backend engineer comfortable in Scala can both build Spark applications without learning a new language first. Section 1.3's word-count comparison is a direct illustration of this ease-of-use gain over hand-written MapReduce.

4.3 Unification — SQL, Streaming, ML, and Graphs in One Engine

Rather than requiring separate specialized systems for SQL querying, stream processing, machine learning, and graph analytics, Spark provides all four as libraries built on the same Spark Core engine, sharing the same underlying RDD/DataFrame execution model, the same cluster manager, and the same cluster resources.

The Unified Spark Stack — One Engine, Many Workloads



The Unified Spark Stack — One Engine, Many Workloads

- Spark SQL & DataFrames — SQL-style querying and a structured, schema-aware API for tabular data.
- Structured Streaming — real-time processing of continuously arriving data, using the same DataFrame API as batch SQL.
- MLlib — distributed machine learning: classification, regression, clustering, and more, trained across a cluster.
- GraphX — graph-parallel processing for network and relationship data (social graphs, recommendation graphs).

Why unification is a genuine competitive advantage, not just convenience

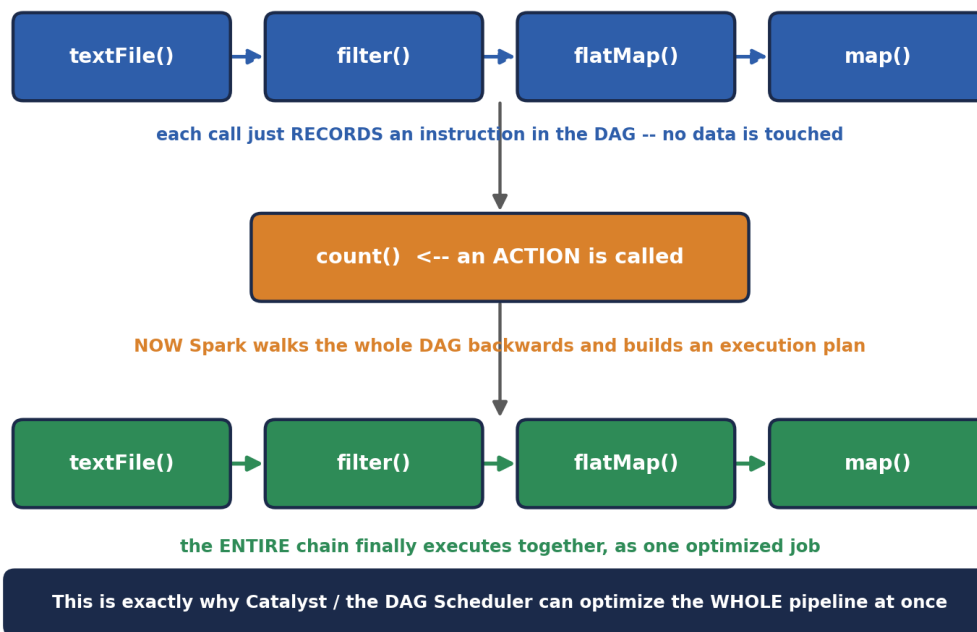
Before Spark, a company might run Hadoop MapReduce for batch ETL, Storm for streaming, Mahout for ML, and a separate graph database for network analysis — four different systems, four different skill sets, four different operational teams, and real friction moving data between them. Spark collapses that into one platform, one team, and one shared cluster, which is a direct and substantial reduction in both engineering headcount and infrastructure cost.

4.4 Lazy Evaluation

Spark deliberately does not execute anything the moment a line of code runs. Operations split into two categories: Transformations (map, filter, groupByKey, and similar) lazily build up a DAG of what SHOULD happen, without touching data; Actions (collect, count, saveAsTextFile, and similar) trigger actual execution, at which point Spark walks the accumulated DAG and executes the necessary transformations as one optimized job.

Lazy Evaluation — Transformations vs. Actions

Building the DAG (transformations only -- nothing executes yet)



Lazy Evaluation — Transformations vs. Actions

```

# PySpark: lazy transformations, then a triggering action
rdd = sc.textFile("hdfs:///data/access.log")
errors = rdd.filter(lambda line: "ERROR" in line)      # transformation --
nothing runs yet
error_words = errors.flatMap(lambda line: line.split()) # transformation --
still nothing runs

count = error_words.count()    # ACTION -- this is the moment Spark actually
executes
print(count)
  
```



The Chef Who Won't Cook Until You Order

A transformation is handing a chef a recipe card. The chef reads it, nods, and sets it on the counter -- nothing gets cooked yet, no matter how many recipe cards you hand over.

*Only when you say 'I'll have that, please' (an action) does the chef look at the **WHOLE** stack of recipe cards at once, plan the most efficient order to execute them, and finally start cooking. This laziness lets Spark see the entire pipeline before committing to a plan, so it can skip unnecessary work and optimize everything together.*

Check yourself — Section 4

- Q1. Name Spark's four unified libraries and, in one phrase each, what problem each one solves.
- Q2. What's the practical performance benefit of Spark waiting until an action is called before executing anything?
- Q3. Why does Spark's multi-language support (Scala/Python/Java/R/SQL) matter for real organizations, beyond just convenience?

Hands-on Challenge:

Write a 4-line PySpark snippet with at least two transformations and one action, then predict, before running it, exactly which line triggers real computation. If you have Spark or Databricks Community Edition available, run it and confirm.

5. Local / Standalone Installation & Environment Setup

Theory is only half of this unit -- the syllabus explicitly calls for hands-on installation of Spark as a standalone user. This section walks through a complete, step-by-step local install on a Linux/POSIX system (the same steps apply nearly unchanged on macOS or WSL on Windows).

5.1 Prerequisites

- A working JDK (Java 8, 11, or 17 depending on your Spark version) — Spark runs on the JVM, per the Prerequisites handbook's memory-model discussion.
- A `JAVA_HOME` environment variable correctly pointing at that JDK installation.
- Enough free disk space (a few hundred MB) and at least 2-4 GB of free RAM for a comfortable local experience.

```
# 1. Check Java is installed and note its version
java -version

# 2. If missing, install a JDK (Ubuntu/Debian example)
sudo apt update && sudo apt install -y openjdk-17-jdk

# 3. Locate the JDK's real install path
readlink -f $(which java)
# e.g. /usr/lib/jvm/java-17-openjdk-amd64/bin/java -> JAVA_HOME is the folder
ABOVE bin/
```

5.2 Downloading & Extracting Spark

```
# 1. Download a pre-built Spark release (choose a version + Hadoop pairing)
cd ~
wget https://downloads.apache.org/spark/spark-3.5.0/spark-3.5.0-bin-hadoop3.tgz

# 2. Extract it into /opt (a conventional home for manually installed software)
sudo tar -xvzf spark-3.5.0-bin-hadoop3.tgz -C /opt/
sudo mv /opt/spark-3.5.0-bin-hadoop3 /opt/spark

# 3. Take ownership so you don't need sudo for everyday use
sudo chown -R $(whoami):$(whoami) /opt/spark
```

5.3 Setting Environment Variables

```
# Add these lines to the end of ~/.bashrc (or ~/.zshrc)
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

```
export SPARK_HOME=/opt/spark
export PATH=$PATH:$JAVA_HOME/bin:$SPARK_HOME/bin:$SPARK_HOME/sbin

# Reload the shell configuration without closing the terminal
source ~/.bashrc

# Verify
echo $SPARK_HOME
spark-shell --version
```

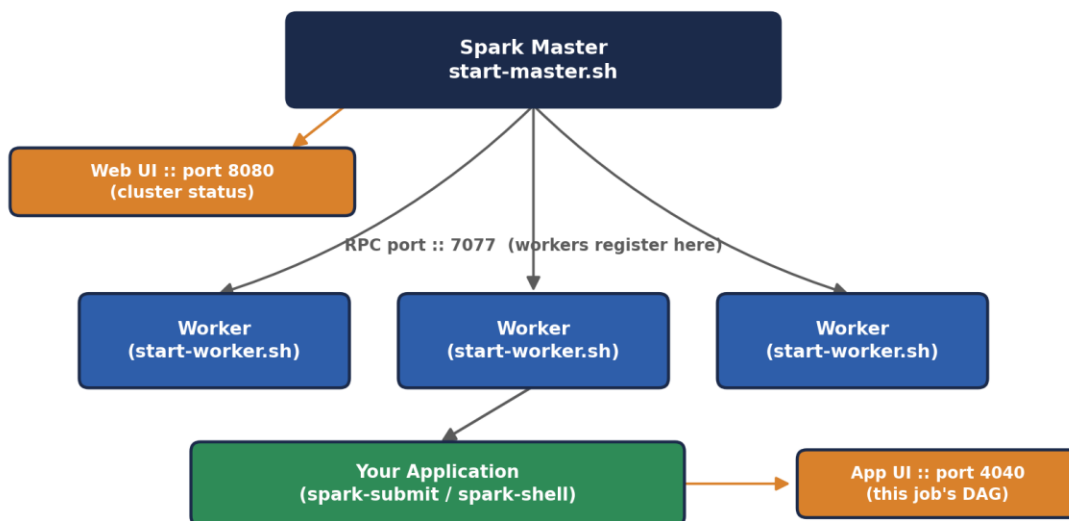
⚠ Common install trap

Setting these variables with `export` only affects your CURRENT terminal session. If you skip adding them to `~/.bashrc` (or your shell's equivalent startup file), `spark-shell` will work now but mysteriously "disappear" the next time you open a new terminal.

5.4 Starting & Stopping the Standalone Cluster

Spark's Standalone cluster manager (Prerequisites: one of several interchangeable cluster manager options) ships with simple shell scripts to start and stop a master and any number of workers on your local machine — exactly the master/worker topology covered conceptually in earlier material, now made concrete.

Spark Standalone Cluster — Ports & Daemons



Spark Standalone Cluster — Ports & Daemons

```
# 1. Start the master daemon
```

```

$SPARK_HOME/sbin/start-master.sh

# 2. Grab the master's URL from its log (or the Web UI, see 5.5)
cat $SPARK_HOME/logs/spark-*-org.apache.spark.deploy.master.Master*.out | grep
'Starting Spark master'
# look for a line containing: spark://<hostname>:7077

# 3. Start one worker, pointing it at the master
$SPARK_HOME/sbin/start-worker.sh spark://<hostname>:7077

# 4. Confirm both daemons are running
jps
# should list: Master, Worker

# 5. When you're done -- stop everything cleanly
$SPARK_HOME/sbin/stop-worker.sh
$SPARK_HOME/sbin/stop-master.sh

```

5.5 Verifying the Web UIs

- Master Web UI — <http://localhost:8080> — shows cluster status: registered workers, available cores/memory, and running applications. This is the first place to check when a job "won't connect."
- Application UI — <http://localhost:4040> — appears only while a specific Spark application (spark-shell, spark-submit, a notebook) is actively running, and shows that job's DAG, stages, and tasks in detail.

```

# Launch the interactive Spark shell against your local standalone cluster
spark-shell --master spark://<hostname>:7077

# While spark-shell is running, open in a browser:
#   http://localhost:8080   -> cluster-wide status
#   http://localhost:4040   -> this shell's own application UI

# Try a one-line sanity check inside spark-shell
scala> val data = sc.parallelize(1 to 100)
scala> data.sum()
// res0: Double = 5050.0

```

A quicker path for pure learning

If you just want to experiment with Spark's APIs without managing a Standalone cluster at all, running ``spark-shell`` or ``pyspark`` with no ``--master`` flag starts Spark in local mode (a single process simulating a tiny cluster on your own machine) — perfect for the code exercises throughout this unit, while the full Standalone setup above is what actually demonstrates the master/worker architecture the syllabus asks for.

 Check yourself — Section 5

Q1. What's the difference between what you'd see on port 8080 versus port 4040?

Q2. Why must JAVA_HOME, SPARK_HOME, and PATH be added to ~/.bashrc rather than just exported once in a terminal?

Q3. What command lets you confirm both the Master and Worker daemons are actually running?

Hands-on Challenge:

On your own machine (or a free cloud shell/VM), complete the full install above end to end: download and extract Spark, set the three environment variables permanently, start a master and one worker, confirm both appear in `jps`, open the port 8080 Web UI in a browser, then launch `spark-shell` against your cluster and run the one-line sanity check shown above.

6. Spark vs. Hadoop Ecosystem

This closing section brings every earlier thread together into one precise, industry-standard comparison matrix. Note the framing from the very first line: this is Spark vs. the MapReduce PROCESSING engine specifically, not vs. Hadoop's storage (HDFS) or resource management (YARN) layers, which Spark commonly continues to use.

Aspect	Hadoop MapReduce	Apache Spark
Storage	HDFS (owns no compute-side storage of its own)	No native storage — reads/writes HDFS, S3, or any compatible store
Compute model	Disk-based, stage-by-stage Map/Reduce	In-memory RDD/DataFrame execution across stages
Speed (iterative jobs)	Baseline — every stage boundary hits disk	Commonly 10-100x faster on multi-pass workloads
Resource management	Classic: JobTracker/TaskTracker. Modern: YARN	Standalone, YARN, or Kubernetes — pluggable
Cost	Lower RAM requirement; leans on disk more	Higher RAM requirement to fully exploit in-memory gains
Real-time support	None natively (batch-only by design)	Native, via Structured Streaming
Machine learning	Mahout (largely superseded)	MLlib, built into the same engine
Graph processing	No native equivalent	GraphX, built into the same engine
Developer experience	Verbose Java Mapper/Reducer/Driver classes	Concise Scala/Python/SQL APIs, lazy DAG optimization



The one sentence to remember

"Spark and Hadoop aren't rivals fighting over the same job — HDFS is commonly the storage foundation Spark computes on top of, YARN is commonly the resource manager Spark runs under, and Spark is the faster, more general-purpose engine that replaced hand-written MapReduce specifically for the processing layer." Said confidently, this single framing outperforms a much longer feature-by-feature answer in an interview setting.



Check yourself — Section 6

Q1. Precisely which layer of the Hadoop ecosystem does Spark replace, and which layers does it typically continue to depend on?

Q2. Name two capabilities Spark has that classic Hadoop MapReduce has no native equivalent for.

Aspect	Hadoop MapReduce	Apache Spark
Q3. Why might a cost-conscious team with mostly simple, single-pass batch jobs still choose plain MapReduce over Spark? Hands-on Challenge: Write your own two-sentence answer to "is Spark a replacement for Hadoop?" as if answering in a technical interview, without looking back at the table above.		

Unit I Review

Quick Review Quiz

Three questions covering the full breadth of this unit. Try to answer each from memory before checking against the sections above.

1. Q1. What specific design choice in classic MapReduce causes its high latency on multi-stage jobs, and what does Spark do differently?
2. Q2. In the Kappa architecture, how is historical data reprocessed without maintaining a separate batch codebase?
3. Q3. Name Spark's four unified libraries built on top of Spark Core, and one real-world use case for each.

Answer key (cover before peeking)

A1: MapReduce writes intermediate results to disk after every stage and pays fresh JVM/resource-negotiation startup costs per job; Spark keeps intermediate RDDs in memory across stages within long-lived Executors, avoiding both costs.

A2: By replaying the stream's log from an earlier point in time through the SAME streaming engine used for live data — no separate batch pipeline is needed.

A3: Spark SQL (BI/analyst querying), Structured Streaming (fraud detection / real-time dashboards), MLlib (recommendation models, churn prediction), GraphX (social network / relationship analysis).

Hands-On Exercise

Complete, end to end, using your local Spark Standalone installation from Section 5:

4. Start your Standalone master and one worker, and confirm both are visible on the port 8080 Web UI.
5. Launch spark-shell (or pyspark) against your cluster URL.
6. Create a small RDD from a list of at least 20 numbers of your choosing, using `sc.parallelize()`.
7. Chain at least two transformations (e.g., `filter()` to keep even numbers, then `map()` to square them) — nothing should execute yet.
8. Call an action (e.g., `collect()` or `sum()`) and observe the result — this is the moment real computation happens.
9. While the job runs, open the port 4040 Application UI in a browser and screenshot (or describe in your own words) the DAG visualization for the job you just ran.

What this exercise is really testing

This single exercise deliberately exercises every syllabus topic in this unit at once: a working Standalone install (Section 5), lazy evaluation in action (Section 4.4), and a firsthand look at the in-memory

execution model (Section 3) that makes Spark faster than the MapReduce approach described in Section 1.